

UNIVERSITÉ MOULAY ISMAIL

Faculté des Sciences de Meknès

Département d'Informatique

Parcours d'Excellence — Master SDIA 2025–2026

Rapport d'État d'Avancement Sprint 2

Agentic GraphRAG pour le Raisonnement Complexe sur des Corpus Techniques

SPRINT 2 — Pipeline
d'Indexation Hybride

Vecteurs + Graphe de Connaissances

05 Mai – 01 Juin 2026

Étudiante : Wiame Anejjar

Encadrant : Pr. Abdelaaziz Hessane

Parcours : Master SDIA 2025–2026

Université : Moulay Ismail — FSM

Date : 07 Juin 2026

Table des matières

Résumé Exécutif	5
1 Étude Approfondie de l'API LightRAG	7
1.1 Présentation générale	7
1.2 Les quatre modes de requêtage	7
1.2.1 Mode naive	7
1.2.2 Mode local	8
1.2.3 Mode global	9
1.2.4 Mode hybrid : Mode principal	10
1.2.5 Tableau comparatif des quatre modes	11
1.3 Paramètres clés de configuration	11
1.4 Fichiers générés par LightRAG	12
2 Pipeline d'Indexation Hybride	14
2.1 Vue d'ensemble du pipeline	14
2.2 Améliorations apportées suite aux remarques du prof	14
2.2.1 Amélioration 1 : Augmentation de <code>max_tokens</code>	14
2.2.2 Amélioration 2 : Format JSON strict des triplets	14
2.2.3 Amélioration 3 : Normalisation canonique des entités	16
2.3 Résolution du problème de compatibilité Ollama	17
2.4 Configuration finale du pipeline	19
2.5 Prompt enrichi pour la qualité du graphe	19
2.6 Mécanisme de checkpoint	20
3 Analyse Statistique du Graphe de Connaissances	21
3.1 Statistiques globales	21
3.2 Visualisation de la structure du graphe	22
3.3 Interprétation des résultats	22
3.4 Exemples de triplets extraits	23
3.4.1 Exemple de triplets extraits par LightRAG	24
3.4.2 Exemple de triplets extraits par SpaCy + REBEL	25

3.4.3	Analyse comparative	25
3.4.4	Limites de la comparaison et pistes d'amélioration	26
3.5	Exploration du graphe dans Neo4j	26
3.6	Pipeline RAG vectoriel et évaluation	28
3.6.1	Description du pipeline	28
3.6.2	Paramètres principaux	29
3.6.3	Principe de génération de réponse	29
3.6.4	Métriques d'évaluation	29
3.6.5	Résultats observés	29
3.6.6	Problématiques identifiées	30
3.6.7	Solutions et améliorations proposées	30
3.6.8	Améliorations techniques proposées (LightRAG)	30

Table des figures

2.1	Contraintes de qualité ajoutées au prompt d'extraction	20
3.1	Analyse statistique du graphe de connaissances généré par LightRAG .	22
3.2	Exemple de triplets extraits par LightRAG	24
3.3	Exemple de triplets extraits par SpaCy + REBEL	25
3.4	Entités les plus connectées du graphe dans Neo4j	27
3.5	Exemple de sous-graphe thématique autour des Large Language Models	28
3.6	Sous-graphe thématique RAG	28

Liste des tableaux

1.1	Comparaison des quatre modes de requêtage LightRAG	11
1.2	Évolution des paramètres LightRAG Sprint 1 → Sprint 2	12
1.3	Fichiers générés par LightRAG après indexation	13
2.1	Configuration finale du pipeline d'indexation Sprint 2	19
3.1	Statistiques globales du graphe de connaissances	21
3.2	Requêtes exploratoires exécutées dans Neo4j	27

Résumé Exécutif

Sprint 2 — Vue d'ensemble

Période : 05 Mai – 01 Juin 2026 (4 semaines)

Thème : Pipeline d'Indexation Hybride : Vecteurs + Graphe de Connaissances

Statut : **En cours**

Ce qui était prévu

Le Sprint 2 avait pour objectif de construire le pipeline d'indexation hybride complet du projet Agentic GraphRAG. Les quatre semaines couvraient :

- (1) l'étude approfondie de l'API LightRAG et l'indexation des 500 documents en mode hybrid avec logging des triplets
- (2) une extraction NER+RE complémentaire avec GLiNER et REBEL pour comparaison qualitative
- (3) le chargement du graphe complet dans Neo4j avec dix requêtes Cypher exploratoires
- (4) l'implémentation du RAG vectoriel baseline (LangChain + ChromaDB) et son évaluation sur 20 questions HotpotQA.

Ce qui a été fait

- Étude approfondie de l'API LightRAG , quatre modes documentés et comparés (naive, local, global, hybrid)
- Amélioration significative du pipeline d'indexation (max_tokens : 300→1800, gleaning : 0→4, prompt enrichi avec contraintes de qualité graphe)
- Résolution d'une incompatibilité critique entre Ollama v0.22.0 et l'endpoint `/v1/chat/completions`
- Indexation de 500 documents avec logging JSON strict des triplets et calcul du taux de parsing

- Implémentation de la normalisation canonique des entités (LLM, GNN, RAG, etc.)
- Extraction NER+RE avec GLiNER + REBEL sur le corpus
- Chargement du graphe dans Neo4j avec requêtes Cypher
- Évaluation quantitative : latence, RAGAS Faithfulness, nœuds traversés

Ce qui reste à faire

- Amélioration de graphe (lightrag)
- Améliorer la comparaison quantitative LightRAG vs GLiNER/REBEL
- Améliorer RAG vectoriel baseline + évaluation HotpotQA

Chapitre 1

Étude Approfondie de l'API LightRAG

1.1 Présentation générale

LightRAG [1] est la bibliothèque centrale du projet. Elle expose une API Python simple (`pip install lightrag-hku`) qui encapsule un pipeline complet de construction de graphe de connaissances et de récupération hybride. Ce chapitre documente les quatre modes de requêtage, les paramètres clés de configuration, et les choix effectués pour ce sprint.

1.2 Les quatre modes de requêtage

L'API LightRAG propose quatre modes de récupération qui diffèrent par leur stratégie de recherche dans les index construits lors de l'indexation.

1.2.1 Mode **naive**

Le mode **naive** correspond à un RAG classique : la question est encodée en vecteur et les k chunks les plus similaires par cosinus sont récupérés depuis la base vectorielle des chunks (`vdb_chunks.json`).

Mode **naive** : Caractéristiques

- **Source** : base vectorielle des chunks
- **Stratégie** : similarité cosinus sur les embeddings de chunks
- **Avantage** : rapide, simple, peu de tokens
- **Limite** : ne tire pas parti du graphe, échoue sur les questions multi-sauts

Listing 1.1 – Exemple de requête en mode naive

```

1 result = await rag.aquery(
2     "What are the main methods for classification?",
3     param=QueryParam(
4         mode='naive',
5         top_k=15,
6         enable_rerank=False
7     )
8 )

```

1.2.2 Mode local

Le mode `local` exploite la structure du graphe de connaissances pour répondre aux questions factuelles précises. Son mécanisme se déroule en trois étapes distinctes.

Étape 1 - Identification des entités requête : La question est encodée en vecteur d'embedding via `nomic-embed-text`. Ce vecteur est comparé par similarité cosinus à tous les vecteurs d'entités stockés dans `vdb_entities.json`. Les k entités les plus proches sémantiquement de la question sont sélectionnées. Par exemple, pour la question *“How does BERT improve NLP tasks?”*, les entités *BERT*, *NLP* et *language model* seront identifiées comme proches.

Étape 2 - Expansion dans le graphe : Pour chaque entité identifiée, LightRAG parcourt le graphe de connaissances (`graph_chunk_entity_relation.graphml`) et récupère toutes ses relations directes (voisins à distance 1). Chaque relation récupérée contient un nœud source, un nœud cible, un prédicat (*uses*, *improves*, *trained_on*) et une description textuelle. Ce sous-graphe local constitue le contexte structuré de la réponse.

Étape 3 - Génération contextuelle : Le LLM reçoit la question, les entités trouvées et leurs relations directes comme contexte, puis génère une réponse précise ancrée dans les faits du graphe.

Mode local : Mécanisme et caractéristiques

- **Sources** : base vectorielle des entités (`vdb_entities.json`) + sous-graphe local (`graph_chunk_entity_relation.graphml`)
- **Stratégie** : similarité cosinus sur les entités → expansion dans le graphe (voisins directs)
- **Avantage** : réponses factuellement précises, ancrées dans les relations explicites du graphe, c'est idéal pour les questions du type *“Quel dataset utilise la méthode X ?”* ou *“Qui a proposé Y ?”*
- **Limite** : le contexte est limité au voisinage immédiat des entités trouvées.

Si les informations nécessaires sont à plus d'un saut dans le graphe, elles ne seront pas récupérées. Échoue sur les questions thématiques nécessitant une vue d'ensemble du corpus.

- **Cas d'usage optimal** : questions factuelles ciblées, questions de type entité-attribut, questions sur les relations directes entre deux concepts précis

1.2.3 Mode global

Le mode **global** adopte une stratégie radicalement différente du mode local. Plutôt que de chercher des entités précises dans le graphe, il interroge les **résumés de relations** pour capturer les thèmes et patterns transversaux du corpus entier.

Étape 1 - Recherche sur les relations résumées : Lors de l'indexation, LightRAG génère pour chaque relation du graphe une description textuelle synthétique (ex. : *“Transformer architectures are widely used for sequence modeling tasks in NLP, particularly for machine translation and text classification”*). Ces descriptions sont encodées en vecteurs et stockées dans `vdb_relationships.json`. En mode global, la question est comparée directement à ces descriptions de relations.

Étape 2 - Agrégation thématique : Les k relations dont les descriptions sont sémantiquement les plus proches de la question sont récupérées. Ces relations proviennent potentiellement de **tous les documents du corpus**, pas seulement d'un sous-graphe local. Cela permet de capturer des tendances globales et des patterns récurrents à travers l'ensemble du corpus, même si aucune entité précise ne correspond à la question.

Étape 3 - Synthèse globale : Le LLM reçoit ces relations thématiquement cohérentes et génère une réponse de synthèse qui reflète les grandes tendances du corpus. Ce mode est particulièrement adapté aux questions ouvertes du type *“Quelles sont les tendances actuelles en apprentissage automatique ?”*

Mode global : Mécanisme et caractéristiques

- **Sources** : base vectorielle des résumés de relations (`vdb_relationships.json`), il couvre l'ensemble du corpus
- **Stratégie** : similarité cosinus entre la question et les descriptions de relations → agrégation des relations thématiquement cohérentes de tout le corpus
- **Avantage** : vision transversale du corpus, excellent pour les questions de synthèse et les tendances générales, et idéal pour *“Quelles méthodes sont les plus utilisées pour la classification ?”*
- **Limite** : peut manquer de précision factuelle. Comme il travaille sur les descriptions de relations et non sur les entités exactes, il peut produire des réponses générales qui omettent des détails spécifiques importants. La réponse reflète des tendances statistiques du corpus plutôt que des faits

précis sur une entité particulière.

- **Cas d’usage optimal** : questions de synthèse thématique, questions sur les tendances générales, questions comparatives à grande échelle (*“Comparer les approches supervisées et non supervisées en NLP”*)

1.2.4 Mode hybrid : Mode principal

Le mode **hybrid** combine les modes **local** et **global** simultanément : il recherche à la fois les entités précises et les thèmes globaux, fusionne les résultats, puis les enrichit avec les chunks de texte les plus proches.

Mode hybrid : Mode choisi pour ce sprint

- **Sources** : entités (local) + relations (global) + chunks (vecteurs)
- **Stratégie** : dual-level retrieval , précision locale ET contexte global simultanément
- **Avantage** : réponses les plus riches, meilleur pour le raisonnement multi-sauts
- **Paramètres utilisés** : `top_k=40`, `chunk_top_k=20`

Listing 1.2 – Requête en mode hybrid : paramètres Sprint 2

```

1 result = await rag.aquery(
2     question,
3     param=QueryParam(
4         mode='hybrid',
5         top_k=40,           # nb entites/rerelations recuperees
6         chunk_top_k=20,    # nb chunks textuels ajoutes
7         enable_rerank=False
8     )
9 )

```

Pourquoi le mode hybrid est supérieur aux deux

Le mode **hybrid** combine exactement les forces des modes **local** et **global** : il récupère simultanément les entités précises du voisinage graphe **ET** les relations thématiques transversales du corpus, puis enrichit le tout avec les chunks de texte les plus proches. Cette triple source d’information produit des réponses à la fois factuellement précises et thématiquement riches, ce qui en fait le mode optimal pour les questions complexes multi-sauts du benchmark HotpotQA.

1.2.5 Tableau comparatif des quatre modes

Critère	naive	local	global	hybrid
Source principale	Chunks teurs	vec- Entités graphe local	+ Relations résu- mées	Entités + Rela- tions + Chunks
Questions simples	Bonne	Bonne	Moyenne	Très bonne
Questions multi-sauts	Faible	Moyenne	Moyenne	Meilleure
Questions synthèse	Moyenne	Faible	Bonne	Bonne
Latence estimée	Faible	Moyenne	Moyenne	Élevée
Tokens consommés	<1K	~2K	~2K	~4K
Usage recommandé	Baseline	Questions fac- tuelles	Questions thé- matiques	Production

TABLE 1.1 – Comparaison des quatre modes de requête LightRAG

1.3 Paramètres clés de configuration

La qualité du graphe généré par LightRAG dépend fortement de plusieurs paramètres de configuration. Le tableau suivant documente les valeurs utilisées au Sprint 1, les problèmes observés, et les valeurs corrigées au Sprint 2.

Paramètre	Sprint 1	Sprint 2	Impact
max_tokens_extract	300	1800	Plus de liberté LLM pour extraire plus d'entités et relations → densité graphe améliorée
entity_extract_max_gle	0	4	Nombre de passes d'extraction par chunk améliore le rappel des entités
chunk_token_size	512	700	Chunks plus grands → plus de contexte par extraction
chunk_overlap	50	120	Overlap plus grand → moins d'entités perdues aux frontières de chunks
max_extract_input_tokens	512	6000	Texte d'entrée plus long → extraction plus complète
entity_types	Non spécifié	11 types	Typage précis des entités : Model, Method, Dataset, Task, Metric, Benchmark, Architecture, Application, Concept, Tool, Organization

TABLE 1.2 – Évolution des paramètres LightRAG Sprint 1 → Sprint 2

1.4 Fichiers générés par LightRAG

Après indexation, LightRAG génère les fichiers suivants dans le répertoire de travail :

Fichier	Contenu
graph_chunk_entity_relation. graphml	Graphe complet au format GraphML (nœuds + arêtes)
vdb_entities.json	Base vectorielle des entités (embeddings)
vdb_relationships.json	Base vectorielle des résumés de relations
vdb_chunks.json	Base vectorielle des chunks de texte
kv_store_text_chunks.json	Texte brut des chunks indexés
kv_store_full_docs.json	Documents complets
kv_store_llm_response_cache. json	Cache des réponses LLM (évite les re- calculs)

TABLE 1.3 – Fichiers générés par LightRAG après indexation

Chapitre 2

Pipeline d'Indexation Hybride

2.1 Vue d'ensemble du pipeline

Le pipeline d'indexation hybride du Sprint 2 est une amélioration significative par rapport au Sprint 1. Il intègre les retours de l'encadrant et résout les problèmes identifiés (faible densité du graphe, format des triplets non contrôlé, endpoint Ollama incompatible).

2.2 Améliorations apportées suite aux remarques du prof

Suite aux remarques de l'encadrant sur le rapport Sprint 1, trois améliorations majeures ont été apportées.

2.2.1 Amélioration 1 : Augmentation de max_tokens

Solution appliquée : augmentation de `max_tokens_extract` de 300 à 1800, et ajout de 4 passes de gleaning (`entity_extract_max_gleaning=4`). Ces deux changements donnent au LLM beaucoup plus de liberté pour extraire exhaustivement toutes les entités et relations présentes dans chaque chunk.

2.2.2 Amélioration 2 : Format JSON strict des triplets

Solution appliquée : implémentation d'une classe `TripletLogger` qui intercepte chaque sortie LLM d'extraction, parse les triplets selon le délimiteur interne de LightRAG (`<|>#<|>`), et calcule en temps réel le taux de parsing réussi.

```
1 @dataclass
2 class TripletLogger:
```

```
3     total_llm_calls: int = 0
4     parse_success: int = 0
5     parse_failed: int = 0
6     missing_complete_delimiter: int = 0
7     malformed_entity_lines: int = 0
8     malformed_relation_lines: int = 0
9     total_entities: int = 0
10    total_relations: int = 0
11    items: list[dict[str, Any]] = field(default_factory=list)
12
13    def parse_extraction(self, text: str, doc_id=None):
14        self.total_llm_calls += 1
15
16        if COMPLETE_DELIM not in text:
17            self.missing_complete_delimiter += 1
18
19        entities, relations = [], []
20
21        try:
22            for line in text.splitlines():
23                parts = line.strip().split(TUPLE_DELIM)
24
25                if parts[0] == "entity":
26                    if len(parts) < 4:
27                        self.malformed_entity_lines += 1
28                        continue
29
30                    entities.append({
31                        "name": parts[1].strip(),
32                        "type": parts[2].strip(),
33                        "description": parts[3].strip()
34                    })
35
36                elif parts[0] == "relation":
37                    if len(parts) < 5:
38                        self.malformed_relation_lines += 1
39                        continue
40
41                    relations.append({
42                        "source": parts[1].strip(),
43                        "target": parts[2].strip(),
```

```

44         "relation_keywords": parts[3].strip(),
45         "description": parts[4].strip()
46     })
47
48     self.parse_success += 1
49     self.total_entities += len(entities)
50     self.total_relations += len(relations)
51
52     except Exception:
53         self.parse_failed += 1

```

Listing 2.1 – Classe TripletLogger améliorée pour le suivi du parsing, la détection des erreurs de format et la journalisation des triplets extraits

2.2.3 Amélioration 3 : Normalisation canonique des entités

Un problème de fragmentation du graphe a été identifié : la même entité apparaissait sous plusieurs formes (*LLM*, *Large Language Model*, *Large Language Models*, *LLMs*) créant des nœuds dupliqués qui réduisaient artificiellement la densité.

Solution appliquée : implémentation d’une table de correspondance canonique appliquée à chaque entité extraite avant insertion dans le graphe.

```

1 CANONICAL_ALIASES = {
2     "llm": "Large_Language_Models",
3     "llms": "Large_Language_Models",
4     "large_language_model": "Large_Language_Models",
5     "vlm": "Vision-Language_Models",
6     "gnn": "Graph_Neural_Networks",
7     "graph_neural_network": "Graph_Neural_Networks",
8     "rag": "Retrieval-Augmented_Generation",
9     "retrieval_augmented_generation":
10         "Retrieval-Augmented_Generation"
11 }
12
13 BAD_ENTITY_NAMES = {
14     "model", "models", "method", "methods",
15     "dataset", "datasets", "task", "tasks",
16     "metric", "metrics", "paper", "study"
17 }
18
19 def canonicalize_entity_name(name: str) -> str:
20     key = normalize_key(name)

```

```

21
22     if key in CANONICAL_ALIASES:
23         return CANONICAL_ALIASES[key]
24
25     return name.strip().title()
26
27 def is_bad_entity(name: str) -> bool:
28     key = normalize_key(name)
29
30     if key in BAD_ENTITY_NAMES:
31         return True
32
33     return False

```

Listing 2.2 – Mécanisme de normalisation canonique et filtrage des entités avant insertion dans le graphe

2.3 Résolution du problème de compatibilité Ollama

Problème critique : Semaine 1

Lors du premier lancement de l'indexation, l'erreur suivante est apparue pour tous les documents : 404 Client Error: Not Found for url: <http://localhost:11434/v1/chat/completions>

Diagnostic : La version Ollama 0.22.0 installée localement ne supporte pas l'endpoint `/v1/chat/completions` (compatible OpenAI), qui n'est disponible qu'à partir de la version 0.5.0+. Le code du pipeline utilisait un mécanisme de fallback en trois étapes qui atteignait systématiquement cet endpoint non supporté.

Solution : Simplification du code LLM pour utiliser uniquement `/api/chat`, l'endpoint natif d'Ollama disponible dans toutes les versions.

```

1 async def local_llm_func(prompt,
2                             system_prompt=None,
3                             history_messages=[],
4                             **kwargs):
5
6     extraction = is_extraction_call(system_prompt)
7
8     r = requests.post(
9         f"{OLLAMA_URL}/api/chat",
10        json={

```

```
11         "model": MODEL_NAME ,
12         "messages": messages ,
13         "stream": False ,
14         "options": {
15             "temperature": 0 ,
16             "num_predict": MAX_TOKENS_EXTRACT ,
17             "num_ctx": OLLAMA_NUM_CTX ,
18         },
19     },
20     timeout=600,
21 )
22
23 r.raise_for_status()
24 out = r.json()["message"]["content"]
25
26 if extraction:
27     out = sanitize_extraction_output(out)
28     triplet_logger.parse_extraction(
29         out,
30         doc_id=CURRENT_DOC_ID
31     )
32
33 return out
```

Listing 2.3 – Fonction d'appel du LLM via l'endpoint natif Ollama /api/chat

2.4 Configuration finale du pipeline

Paramètre	Valeur	Justification
MODEL_NAME	llama3.1 :8b	Modèle local Ollama : bon équilibre qualité/vitesse
EMBED_MODEL	nomic-embed-text	768 dimensions : validé au Sprint 1
chunk_token_size	700	Contexte suffisant par chunk
chunk_overlap	120	17% d'overlap : contexte préservé aux frontières
entity_extract_max_gleaning	4	4 passes d'extraction par chunk : maximise le rappel
max_tokens_extract	1800	Liberté maximale pour l'extraction (était 300)
max_extract_input_tokens	6000	Texte d'entrée complet : pas de troncature
OLLAMA_NUM_CTX	8192	Fenêtre de contexte du LLM
entity_types	11 types	Model, Method, Dataset, Task, Metric, Benchmark, Architecture, Application, Concept, Tool, Organization
NB_DOCS	500	Corpus complet arXiv CS.AI

TABLE 2.1 – Configuration finale du pipeline d'indexation Sprint 2

2.5 Prompt enrichi pour la qualité du graphe

En complément des paramètres de configuration, le prompt système d'extraction de LightRAG a été enrichi avec des contraintes spécifiques pour améliorer la qualité des entités et relations extraites.

```

# Graph quality rules: fewer duplicate nodes, more useful cross-document relations.
PROMPTS['entity_extraction_system_prompt'] += """
---Additional Graph Quality Rules---
Extract reusable scientific entities, not full paper titles.
Do not extract the complete document title as an entity unless it is a named model, dataset, benchmark, tool, or method.

Use canonical names consistently:
- LLM, LLMs, Large Language Model, Large Language Models -> Large Language Models
- VLM, VLMs, Vision-Language Model(s) -> Vision-Language Models
- GNN, GNNs, Graph Neural Network(s) -> Graph Neural Networks
- RAG, Retrieval Augmented Generation -> Retrieval-Augmented Generation

Avoid isolated entities. Every extracted entity should participate in at least one relationship.
Prefer fewer high-quality entities over many weak entities.
For each chunk, extract as many meaningful relationships as possible between methods, models, datasets, tasks, metrics, benchmarks, applications, and concepts.
Use specific predicates such as uses, improves, evaluates_on, trained_on, applied_to, part_of, based_on, compares_with, extends, addresses.
Avoid generic predicates like related_to.
"""

except Exception:
    TUPLE_DELIM = '<|>'
    COMPLETE_DELIM = '<|COMPLETE|>'

```

FIGURE 2.1 – Contraintes de qualité ajoutées au prompt d'extraction

2.6 Mécanisme de checkpoint

L'indexation de 500 documents avec un LLM local prend plusieurs heures et jours. Un mécanisme de checkpoint sauvegarde la progression après chaque document, permettant une reprise automatique sans perte en cas d'interruption.

Listing 2.4 – Mécanisme de checkpoint - reprise automatique

```

1 for i in tqdm(range(start, len(docs)), desc='Indexation'):
2     doc_id = docs[i].get('id', str(i))
3     text    = docs[i].get('text', '')
4
5     CURRENT_DOC_ID = doc_id    # pour le TripletLogger
6     try:
7         await rag.ainsert(text, ids=doc_id)
8         success += 1
9     except Exception as e:
10        errors.append({'index': i, 'doc_id': doc_id,
11                       'error': str(e)[:300]})
12        await asyncio.sleep(2)
13    finally:
14        # Sauvegarde apres CHAQUE document
15        save_checkpoint(CHECKPOINT, {
16            'last_index': i + 1,
17            'success': success,
18            'errors': errors
19        })
20    await asyncio.sleep(SLEEP_BETWEEN_DOCS_S)

```

Chapitre 3

Analyse Statistique du Graphe de Connaissances

Afin d'évaluer la qualité structurelle du graphe construit par LightRAG, une analyse statistique a été réalisée sur le fichier `graph_chunk_entity_relation.graphml` généré après l'indexation du corpus de 500 articles arXiv.

3.1 Statistiques globales

Métrique	Valeur
Nombre de nœuds (entités)	5065
Nombre d'arêtes (relations)	5068
Densité du graphe	0.0004
Degré moyen	2.00
Degré médian	1.00
Degré minimum	0
Degré maximum	215
Plus grande composante connexe	2195 nœuds (43.3%)
Coefficient de clusterisation moyen	0.0801
Diamètre	Non calculable (graphe non connexe)

TABLE 3.1 – Statistiques globales du graphe de connaissances

3.2 Visualisation de la structure du graphe

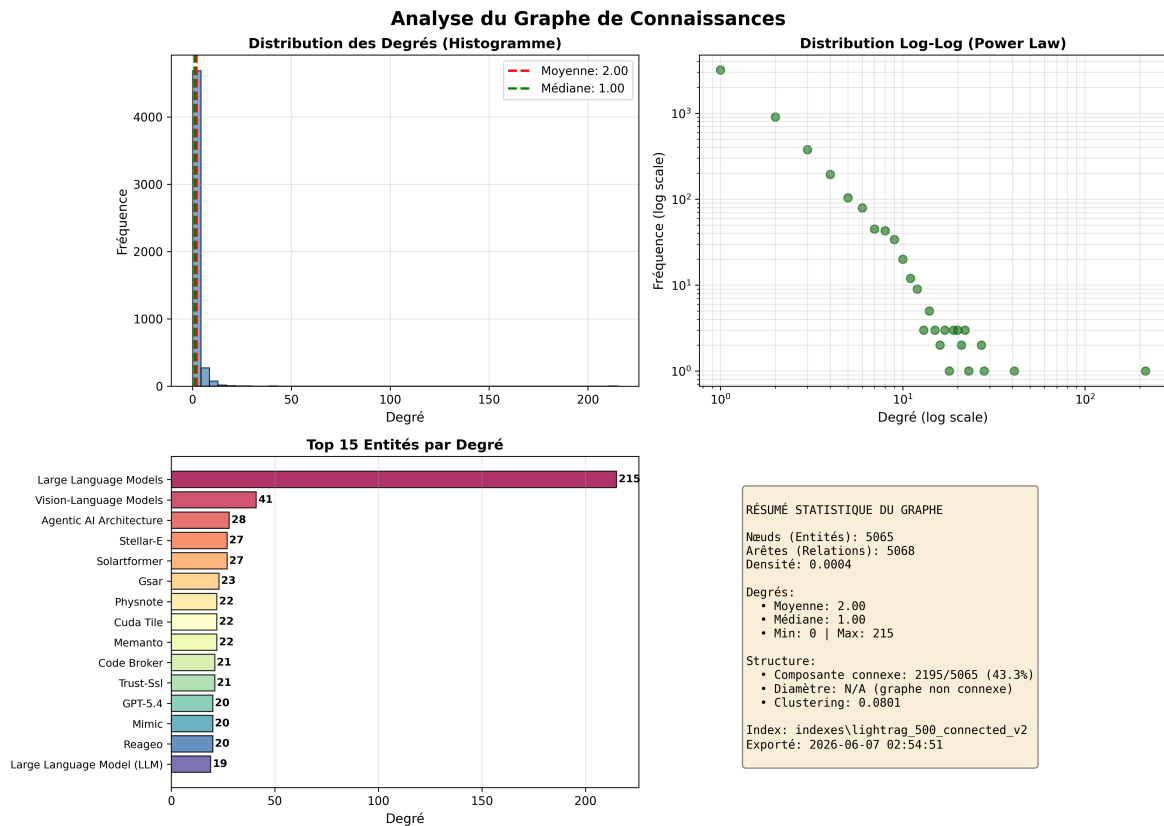


FIGURE 3.1 – Analyse statistique du graphe de connaissances généré par LightRAG

3.3 Interprétation des résultats

Le graphe construit contient **5065 entités** reliées par **5068 relations**. Le nombre d'arêtes est très proche du nombre de nœuds, ce qui indique une structure relativement clairsemée.

La **densité** du graphe est de **0.0004**. Cette valeur très faible signifie que seule une infime partie des connexions théoriquement possibles entre les entités existe réellement. Ce comportement est attendu pour un graphe de connaissances extrait automatiquement à partir d'un corpus scientifique, où chaque entité n'est reliée qu'à un nombre limité de concepts pertinents.

Le **degré moyen** est égal à **2.00**. En moyenne, chaque entité est connectée à deux autres entités du graphe. Cela montre que la majorité des relations extraites sont locales et spécialisées.

La **médiane du degré** est égale à **1.00**. Cela signifie que plus de la moitié des entités possèdent au plus une seule connexion. La différence entre la moyenne (2.00) et la médiane (1.00) indique la présence de quelques nœuds très connectés qui augmentent

artificiellement la moyenne.

Le **degré maximum** observé est de **215**, correspondant à l'entité *Large Language Models*. Cette entité agit comme un hub central reliant un grand nombre de concepts présents dans le corpus.

La plus grande **composante connexe** contient **2195 nœuds**, soit **43.3%** du graphe total. Une composante connexe représente un ensemble d'entités reliées entre elles par un chemin. Le fait que moins de la moitié des nœuds appartiennent à la composante principale montre que le graphe reste fragmenté en plusieurs sous-graphes thématiques indépendants.

Le **coefficient de clusterisation moyen** vaut **0.0801**. Cette mesure évalue la tendance des voisins d'un nœud à être également reliés entre eux. Une valeur faible mais non nulle indique l'existence de petits groupes thématiques cohérents sans pour autant former des communautés fortement denses.

Enfin, le diamètre global n'a pas pu être calculé car le graphe n'est pas entièrement connexe. Certaines entités sont isolées ou appartiennent à des composantes séparées.

3.4 Exemples de triplets extraits

Afin d'illustrer les informations utilisées pour construire le graphe de connaissances, cette section présente des exemples de triplets extraits par les deux approches étudiées : LightRAG et le pipeline SpaCy + REBEL.

3.4.1 Exemple de triplets extraits par LightRAG

```

"triplets": [
  {
    "relations": [
      {
        "source": "Pattern-Based Knowledge Components",
        "target": "Student Code Submissions",
        "relation_keywords": "dataset analysis,knowledge extraction",
        "description": "Student code submissions are analyzed to extract pattern-based knowledge components",
      },
      {
        "source": "Generative Model",
        "target": "Pattern-Based Knowledge Components",
        "relation_keywords": "content generation,model conditioning",
        "description": "The generative model is conditioned on pattern-based knowledge components to generate content",
      },
      {
        "source": "Adaptive Programming Practice",
        "target": "Personalized Worked Example Generation From Student Code Submissions Using Pattern-Based Knowledge Components",
        "relation_keywords": "concept application,educational method",
        "description": "This concept is applied in the context of adaptive programming practice for personalized learning",
      },
      {
        "source": "Expert Evaluation",
        "target": "Generative Model",
        "relation_keywords": "metric evaluation,model assessment",
        "description": "Expert evaluation assesses the performance of the generative model in terms of content quality and relevance",
      },
    ]
  }
]

```

FIGURE 3.2 – Exemple de triplets extraits par LightRAG

Les résultats produits par LightRAG sont organisés en deux parties : une liste d'**entités** et une liste de **relations**. Chaque relation est représentée par un couple (source, cible) enrichi par des mots-clés relationnels (**relation_keywords**) ainsi qu'une description textuelle complète expliquant le lien entre les deux entités. Cette structure fournit un contexte sémantique riche qui facilite la construction d'un graphe de connaissances interprétable et exploitable pour le raisonnement multi-sauts.

3.4.2 Exemple de triplets extraits par SpaCy + REBEL

```
"triplets": [  
  {  
    "head": "AST",  
    "type": "use",  
    "tail": "AST-based analysis"  
  },  
  {  
    "head": "AST-based analysis",  
    "type": "uses",  
    "tail": "AST"  
  },  
  {  
    "head": "example generation",  
    "type": "subclass of",  
    "tail": "learning"  
  },  
  {  
    "head": "personalized learning",  
    "type": "subclass of",  
    "tail": "learning"  
  }  
]
```

FIGURE 3.3 – Exemple de triplets extraits par SpaCy + REBEL

Dans l'approche SpaCy + REBEL, les relations sont directement représentées sous la forme classique de triplets (**head**, **relation**, **tail**). Chaque triplet relie une entité source (**head**) à une entité cible (**tail**) à travers un prédicat (**type**) tel que *use*, *subclass of* ou *part of*. Contrairement à LightRAG, les triplets ne contiennent pas de description textuelle détaillée; l'information relationnelle est uniquement portée par le prédicat extrait.

3.4.3 Analyse comparative

L'exemple présenté montre que LightRAG produit des relations plus riches sémantiquement grâce à l'ajout d'une description explicative et de mots-clés relationnels. À l'inverse, SpaCy + REBEL génère des triplets plus compacts et plus proches de la représentation classique des graphes de connaissances.

3.4.4 Limites de la comparaison et pistes d'amélioration

La comparaison entre LightRAG et l'approche SpaCy + REBEL doit être interprétée avec prudence. En effet, les deux systèmes ne produisent pas exactement le même type de relations.

Dans SpaCy + REBEL, les connaissances sont représentées directement sous forme de triplets explicites (**head**, **relation**, **tail**), ce qui facilite leur comptage et leur analyse. En revanche, LightRAG construit un graphe à partir d'entités et de relations enrichies par des descriptions textuelles et des mots-clés sémantiques. Les relations ne correspondent donc pas toujours à des triplets classiques directement comparables.

Cette différence de représentation peut influencer certaines métriques du graphe, notamment le nombre de relations, la distribution des degrés ou encore la connectivité globale. Ainsi, les résultats obtenus ne reflètent pas uniquement la qualité de l'extraction mais également les choix de modélisation propres à chaque approche.

Pour obtenir une comparaison plus rigoureuse, plusieurs améliorations sont envisagées :

- convertir les relations LightRAG vers un format standard (**sujet**, **relation**, **objet**) avant l'évaluation ;
- appliquer les mêmes règles de normalisation et de fusion d'entités aux deux graphes ;
- évaluer les systèmes sur un jeu de données annoté contenant des relations de référence (*ground truth*) ;
- comparer la qualité sémantique des relations extraites en complément des métriques purement structurelles.

Ces améliorations permettraient d'obtenir une comparaison plus objective entre les deux méthodes d'extraction de connaissances.

3.5 Exploration du graphe dans Neo4j

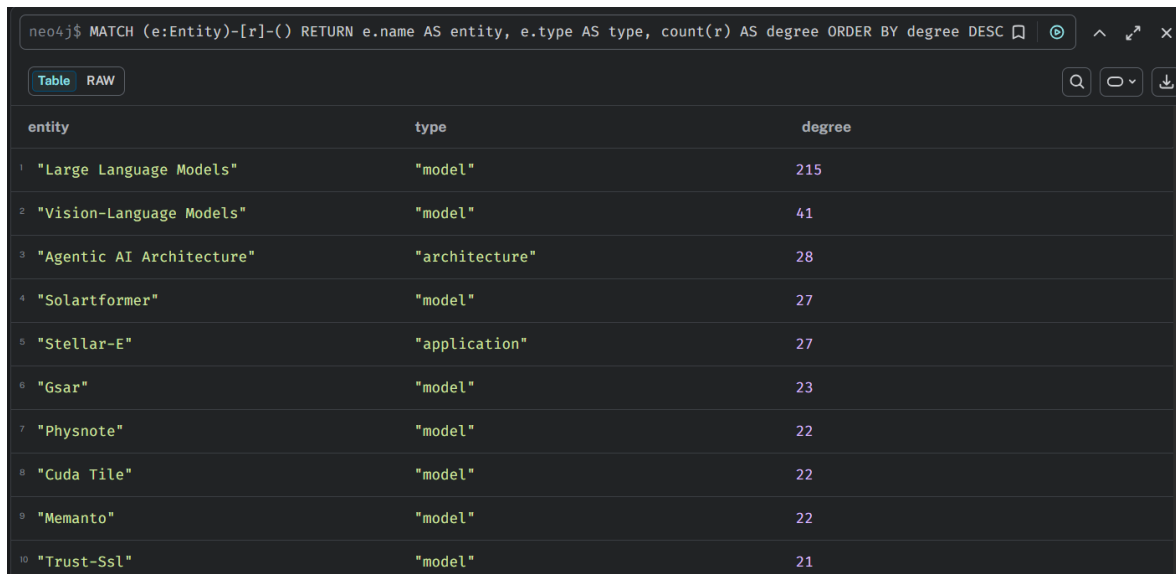
Après la construction du graphe de connaissances avec LightRAG, celui-ci a été exporté vers Neo4j afin de faciliter son exploration et son analyse. Neo4j permet d'exécuter des requêtes Cypher pour examiner la structure du graphe, identifier les entités importantes, explorer les relations et détecter des communautés thématiques.

Dans ce cadre, dix requêtes exploratoires ont été réalisées conformément aux objectifs du projet.

N°	Objectif de la requête
1	Identifier les entités les plus connectées du graphe
2	Analyser la fréquence des types de relations
3	Détecter les entités isolées
4	Identifier les paires d'entités les plus reliées
5	Rechercher le plus court chemin entre deux concepts
6	Explorer le sous-graphe thématique des LLM
7	Explorer le sous-graphe thématique du RAG
8	Explorer le sous-graphe thématique des Graph Neural Networks
9	Calculer les statistiques globales du graphe
10	Détecter automatiquement les communautés à l'aide de l'algorithme Louvain

TABLE 3.2 – Requêtes exploratoires exécutées dans Neo4j

Les Figures suivantes présentent quelques exemples de résultats obtenus dans Neo4j, notamment les entités les plus connectées, les sous-graphes thématiques ainsi que les communautés détectées automatiquement.



The screenshot shows a Neo4j interface with a Cypher query: `neo4j$ MATCH (e:Entity)-[r]-() RETURN e.name AS entity, e.type AS type, count(r) AS degree ORDER BY degree DESC`. The results are displayed in a table with columns 'entity', 'type', and 'degree'. The top 10 results are as follows:

	entity	type	degree
1	"Large Language Models"	"model"	215
2	"Vision-Language Models"	"model"	41
3	"Agentic AI Architecture"	"architecture"	28
4	"Solartformer"	"model"	27
5	"Stellar-E"	"application"	27
6	"Gsar"	"model"	23
7	"Physnote"	"model"	22
8	"Cuda Tile"	"model"	22
9	"Memanto"	"model"	22
10	"Trust-Ssl"	"model"	21

FIGURE 3.4 – Entités les plus connectées du graphe dans Neo4j

```

1 MATCH (a:Entity)-[r:RELATES_TO]-(b:Entity)
2 WHERE toLower(a.name) CONTAINS 'llm'
3 OR toLower(a.name) CONTAINS 'language model'
4 OR toLower(b.name) CONTAINS 'llm'
5 OR toLower(b.name) CONTAINS 'language model'
6 RETURN a.name AS source, r.type AS relation, b.name AS target
7 LIMIT 80;

```

source	relation	target
"Clinical Field Recovery"	"information recovery,task contribution"	"LLM-guided Adaptive Policy"
"LLM-guided Adaptive Policy"	"information recovery,task contribution"	"Clinical Field Recovery"
"LLM-guided Adaptive Policy"	"method evaluation,performance comparison"	"Question Selection Benchmark"
"Question Selection Benchmark"	"method evaluation,performance comparison"	"LLM-guided Adaptive Policy"
"Prompt Reframings"	"concept application,model evaluation"	"Reliability Auditing for Downstream LLM tasks in Psychiatry: LLM-Generated Hospitalization Risk Scores"
"Reliability Auditing for Downstream LLM tasks in Psychiatry: LLM-Generated Hospitalization Risk Scores"	"concept application,model evaluation"	"Predicted Hospitalization Risk Scores"
"Reliability Auditing for Downstream LLM tasks in Psychiatry: LLM-Generated Hospitalization Risk Scores"	"concept application,model evaluation"	"Prompt Reframings"

FIGURE 3.5 – Exemple de sous-graphe thématique autour des Large Language Models

```

neo4j$ MATCH (a:Entity)-[r:RELATES_TO]-(b:Entity) WHERE toLower(a.name) CONTAINS 'retrieval' OR toLower(a.name)

```

source	relation	target
"Complex Eligibility Criteria"	"clinically meaningful signals,information selection"	"Retrieval-Augmented Generation"
"End-to-End LLM Approaches"	"computational cost,performance comparison"	"Lightweight Retrieval-Augmented Generation And Large Language Model-Based Modeling"
"Lightweight Retrieval-Augmented Generation And Large Language Model-Based Modeling"	"computational cost,performance comparison"	"End-to-End LLM Approaches"
"Lightweight Retrieval-Augmented Generation And Large Language Model-Based Modeling"	"benchmarking,evaluation"	"N2C2 Dataset, Sigir Dataset, Trec 2021/2022 Dataset"
"Lightweight Retrieval-Augmented Generation And Large Language Model-Based Modeling"	"computational cost,scalability"	"Patient-Trial Matching"
"Lightweight Retrieval-Augmented Generation And Large Language Model-Based Modeling"	"benchmarking,evaluation"	"N2C2 Dataset, Sigir Dataset, Trec 2021/2022 Dataset"

FIGURE 3.6 – Sous-graphe thématique RAG

3.6 Pipeline RAG vectoriel et évaluation

3.6.1 Description du pipeline

Un pipeline de RAG vectoriel a été implémenté afin d'établir une baseline de comparaison avec l'approche GraphRAG/LightRAG. Le système repose sur une base de données vectorielle ChromaDB alimentée par des embeddings générés via Ollama.

Le pipeline est structuré comme suit :

Corpus → Chunking → Embeddings → ChromaDB → Retrieval → LLM

Les documents du corpus (500 abstracts arXiv) sont transformés en objets LangChain, puis découpés en segments textuels avant d'être indexés dans ChromaDB.

3.6.2 Paramètres principaux

- Modèle d'embedding : `nomic-embed-text`
- LLM : `llama3.1:8b` (Ollama)
- Chunk size : 900 tokens
- Overlap : 120 tokens
- Top-k retrieval : 5 documents
- Contexte LLM : 8192 tokens
- Dataset d'évaluation : HotpotQA (20 échantillons)

3.6.3 Principe de génération de réponse

Pour chaque question, le système :

1. récupère les k chunks les plus similaires via recherche vectorielle,
2. concatène ces chunks en contexte,
3. génère une réponse avec un LLM local (zero temperature),
4. applique une évaluation automatique via RAGAS.

3.6.4 Métriques d'évaluation

Le système a été évalué avec le framework RAGAS selon quatre métriques :

- Faithfulness : cohérence de la réponse avec le contexte,
- Answer Relevancy : pertinence de la réponse,
- Context Precision : qualité des passages récupérés,
- Context Recall : couverture des informations nécessaires.

3.6.5 Résultats observés

Les tests sur 20 questions HotpotQA montrent que :

- le retrieval fonctionne correctement dans un cadre mono-hop,
- les performances chutent fortement sur les questions multi-hop,
- plusieurs cas de *retrieval failure* ont été détectés,
- la plus part des réponses sont générées sans contexte suffisant.

Le principal problème identifié est que le dataset HotpotQA repose sur des documents Wikipedia, alors que le système est indexé sur un corpus arXiv personnalisé, ce qui entraîne un désalignement entre les *ground truths* et les documents récupérés.

3.6.6 Problématiques identifiées

Les limitations suivantes ont été observées lors des expérimentations :

- **Mismatch de domaine** : HotpotQA n'est pas aligné avec le corpus PFE, rendant le recall artificiellement nul dans certains cas.
- **Limite du retrieval vectoriel** : incapacité à gérer efficacement les requêtes multi-hop complexes.
- **Dépendance au top-k** : une valeur faible ($k=5$) limite la couverture contextuelle.

3.6.7 Solutions et améliorations proposées

Afin de corriger ces limitations, plusieurs améliorations ont été envisagées :

- construction d'un dataset d'évaluation basé sur le corpus arXiv (questions + ground truth internes),
- augmentation dynamique du paramètre top-k pour améliorer la couverture,
- remplacement du benchmark HotpotQA par un dataset aligné au domaine scientifique du corpus.

3.6.8 Améliorations techniques proposées (LightRAG)

Plusieurs optimisations au niveau de la chaîne LightRAG ont été identifiées afin de corriger les limites observées lors de l'évaluation :

- **Alignement des embeddings** : vérification et harmonisation de la fonction d'embedding entre l'indexation et la phase de retrieval afin d'éviter les cas de *no-context*.
- **Optimisation de l'index vectoriel** : contrôle de l'intégrité de l'index (chunks, entités, relations) et détection d'éventuelles erreurs silencieuses lors de l'insertion dans la base vectorielle.
- **Amélioration des paramètres de retrieval** : augmentation de *top-k* et *chunk_top_k*, ainsi que l'ajustement du seuil de similarité pour améliorer la couverture contextuelle.
- **Standardisation des relations** : introduction d'un ensemble limité de prédicats relationnels afin de réduire la variabilité et améliorer la cohérence du graphe sémantique.

- **Canonicalisation des entités** : normalisation des noms d'entités pour réduire les doublons et améliorer l'overlap entre différents extracteurs.
- **Renforcement du prompt d'extraction** : utilisation de prompts contraints pour limiter les relations verbeuses et encourager des triplets structurés et cohérents.
- **Réduction de la génération libre** : limitation du nombre de tokens générés lors de l'extraction afin de réduire les hallucinations et améliorer la qualité des triplets extraits.

Remarque

Je suis actuellement en cours d'investigation pour surmonter le blocage rencontré lors des expérimentations, notamment concernant les limitations du système de retrieval et l'évaluation du graphe.

Je serais également reconnaissante pour toute remarque ou suggestion concernant les pistes d'amélioration proposées ainsi que les choix techniques effectués à ce stade, afin d'orienter au mieux la suite du travail.

Bibliographie

- [1] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG : Simple and Fast Retrieval-Augmented Generation,” *arXiv preprint arXiv :2410.05779*, 2024.
- [2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From Local to Global : A Graph RAG Approach to Query-Focused Summarization,” *arXiv preprint arXiv :2404.16130*, 2024.
- [3] Y. Gao et al., “Retrieval-Augmented Generation for Large Language Models : A Survey,” *arXiv preprint arXiv :2312.10997*, 2023.
- [4] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS : Automated Evaluation of Retrieval Augmented Generation,” *arXiv preprint arXiv :2309.15217*, 2023.